Estimating Parameters of Structural Models Using Neural Networks

Bhatt, Misra, and Qi (2023)

Tate Mason

John Munro Godfrey Sr. Dept. of Economics
University of Georgia

March 23, 2026

Roadmap

What Is a Neural Network?

- A **neural network** is a parameterized function $f_\phi : \mathbb{R}^K \to \mathbb{R}^P$ built as a composition of affine transformations and nonlinearities
- Organized into *layers*: input $\to$ hidden $\to$ output
- Each hidden layer ℓ computes

$$h^{(\ell)} = \sigma\left(W^{(\ell)}\, h^{(\ell-1)} + b^{(\ell)}\right)$$

  where σ is an *activation function* applied element-wise
- Networks are **trained** by minimizing a loss over labeled examples $\{(x_s, y_s)\}_{s=1}^{S}$ via stochastic gradient descent
- **Universal approximation**: a sufficiently wide (or deep) network can approximate any continuous function on a compact domain

Machine Learning in Economics

- ML methods are increasingly used in empirical economics
  - Prediction problems: Mullainathan & Spiess (2017)
  - Causal inference: Athey & Imbens (2019)
- In **structural** work, the core challenge is *computational*: many models lack closed-form likelihoods
- Standard simulation-based approaches:
  1. **Simulated MLE (SMLE)**: approximate likelihood via simulation
  2. **Simulated Method of Moments (SMM)**: match simulated to data moments
  3. **Approximate Bayesian Computation (ABC)**: accept/reject by moment distance
- All require **many model simulations per evaluation** of the objective $\Rightarrow$ estimation is expensive for complex models

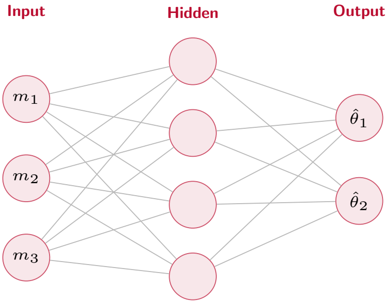
Neural Networks for Structural Estimation

- **Key insight**: if simulation is cheap, train a NN to *invert* the mapping from parameters to observable moments
- The estimator is built in two phases:
  1. **Offline (once):** simulate S datasets across parameter space, compute moments, train NN $\hat{f} : m \mapsto \theta$
  2. **Online (fast):** given observed data, compute moments m^{obs}, then $\hat{\theta} = \hat{f}(m^{\text{obs}})$ — one forward pass
- Separates the **expensive** simulation (offline) from **fast** estimation (online)
- Especially valuable when the model is estimated repeatedly or applied to many datasets
- This paper: theoretical foundations + practical guidelines + applications

Roadmap

Network Architecture

- A feedforward NN has an **input layer**, one or more **hidden layers**, and an **output layer**
- **Depth**: number of hidden layers
- **Width**: number of units per layer
- Total parameters: $\sum_{\ell} (d_{\ell-1} \cdot d_\ell + d_\ell)$ where d_ℓ is the width of layer ℓ
- Complexity grows quickly with depth and width



Neuron Types and Complexity

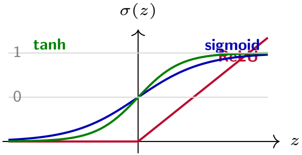
- **Input neurons**: receive raw features (e.g., data moments m)
- **Hidden neurons**: learn intermediate representations
  - Each unit: $h_j = \sigma(w_j \cdot h_{\text{prev}} + b_j)$
  - Activation σ introduces the nonlinearity needed to approximate complex mappings
- **Output neurons**: produce the final estimate $\hat{\theta}$ (no activation in regression)

Model complexity is governed by:

- Depth: deeper networks learn more abstract representations
- Width: wider networks have more capacity per layer
- Larger networks risk overfitting $\Rightarrow$ need regularization (L2, dropout, early stopping)

Activation Functions: Visual Intuition

Why nonlinearity? Without σ, stacking layers collapses to one linear map — no expressive power gained.



- **ReLU** $= \max(0, z)$
  Fast, sparse; default for deep nets
- **Sigmoid** $= (1 + e^{-z})^{-1}$
  Squashes to $(0, 1)$; good for probabilities
- **tanh**
  Zero-centered; often preferred over sigmoid in hidden layers
- **NNE**: ReLU hidden, **no activation** on output

Functions: Training, Loss, and Others

Loss function (what we minimize):

- NNE regression: mean squared error

$$\mathcal{L}(\phi) = \frac{1}{S}\sum_{s=1}^{S}\left\| f_\phi(m^{(s)}) - \theta^{(s)} \right\|^2$$

- Classification: cross-entropy $\mathcal{L} = -\frac{1}{S}\sum_s \sum_k y_k^{(s)} \log \hat{p}_k^{(s)}$

Other key functions:

- **Activation functions**: ReLU $= \max(0, z)$, sigmoid $= (1 + e^{-z})^{-1}$, tanh
- **Regularization**: L2 weight decay, dropout (randomly zero units during training)
- **Batch normalization**: rescale layer inputs for stable training
- **Learning rate schedule**: reduce step size over epochs

Training Process

1. **Forward pass**: compute $\hat{\theta} = f_\phi(m)$ layer by layer
2. **Compute loss**: $\mathcal{L}(\phi) = \|\hat{\theta} - \theta\|^2$
3. **Backward pass** (backpropagation): apply chain rule to get $\nabla_\phi \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial h^{(\ell)}} \cdot \sigma'\left(W^{(\ell)} h^{(\ell-1)}\right) \cdot \left(h^{(\ell-1)}\right)^\top$$

4. **Parameter update**: stochastic gradient descent (or Adam)

$$\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}$$

5. Repeat over *mini-batches* and *epochs* until convergence

- Learning rate η is the most important hyperparameter
- Validation set used to monitor overfitting and trigger early stopping

Roadmap

AR(1) Model Setup

Data generating process:

$$y_t = \rho\, y_{t-1} + \varepsilon_t, \quad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$$

- Parameters to estimate: $\theta = (\rho,\, \sigma)$, with $|\rho| < 1$
- Observed: time series $\{y_t\}_{t=1}^{T}$
- **Moments** that summarize the data:

$$m(y) \;=\; \left(\widehat{\mathrm{Var}}(y),\; \hat{\rho}_1,\; \hat{\rho}_2\right)$$

  where $\hat{\rho}_k$ is the sample autocorrelation at lag k
- MLE is available for AR(1) — NNE is used here as a *proof of concept* to validate accuracy against a known benchmark

AR(1): Simulate Data and Compute Moments

Algorithm, Steps 1–2:

1. Specify prior $\Pi(\theta)$ over (ρ, σ)
2. For each simulation $s = 1, \ldots, S$:
   - Draw $\theta^{(s)} \sim \Pi$
   - Simulate AR(1) time series of length T
   - Compute moments $m^{(s)} = m\left(\{y_t^{(s)}\}\right)$

Moments carry the information about (ρ, σ); the NN learns to invert this mapping.

```
import numpy as np

def simulate_ar1(rho, sigma, T=200, rng=None):
    rng = rng or np.random.default_rng()
    y = np.zeros(T)
    for t in range(1, T):
        y[t] = rho * y[t-1] \
             + sigma * rng.standard_normal()
    return y

def compute_moments(y):
    v  = np.var(y)
    r1 = np.corrcoef(y[:-1], y[1:])[0, 1]
    r2 = np.corrcoef(y[:-2], y[2:])[0, 1]
    return np.array([v, r1, r2])
```

AR(1): Training the NNE

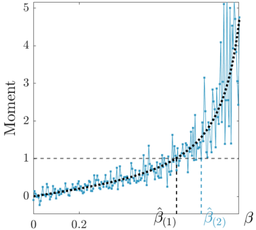
Algorithm, Steps 3–4:

3. Train NN $f_\phi : m \mapsto \theta$ on $\{(m^{(s)}, \theta^{(s)})\}_{s=1}^{S}$
4. **Estimation:** given $\{y_t^{\text{obs}}\}$, compute m^{obs}, then $\hat{\theta} = f_\phi(m^{\text{obs}})$

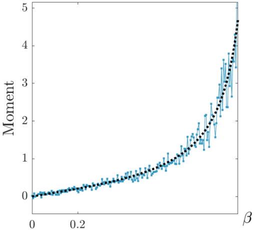
Steps 1–3 are **offline** (one-time cost).
Step 4 is **online**: milliseconds per dataset.

```
from sklearn.neural_network import MLPRegressor

rng, S = np.random.default_rng(42), 5_000

rho_s   = rng.uniform(-0.95, 0.95, S)
sigma_s = rng.uniform(0.1,   2.0,  S)
thetas  = np.column_stack([rho_s, sigma_s])

moments = np.array([
    compute_moments(
        simulate_ar1(r, s, rng=rng))
    for r, s in thetas
])

nne = MLPRegressor(
    hidden_layer_sizes=(64, 64),
    max_iter=500, random_state=42)
nne.fit(moments, thetas)
```

AR(1) Results

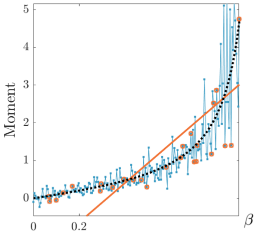


(a) Black dotted curve shows $\mathbb{E}(y_t y_{t-1})$ as a function of β. Jagged blue curve shows the approximation by SMM with $R = 1$.

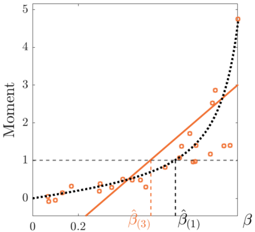


(b) Jagged blue curve shows the approximation by SMM with $R = 5$.

AR(1) Results (cont.)

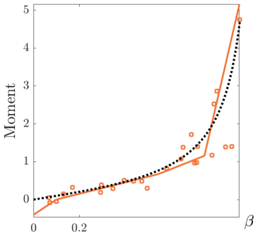


(c) A linear fit with $L = 25$ training points sampled from jagged blue curve. These training points are shown as red circles.

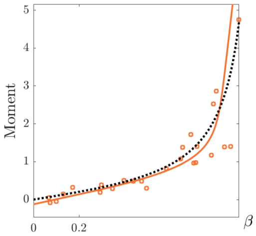


(d) Estimation of β based on the linear fit.

AR(1) Results (cont.)



(e) A fit by neural net with ReLu activation, using $L = 25$ training points.



(f) A fit by neural net with sigmoid activation, using $L = 25$ training points.

Figure 2: Estimation of AR(1)

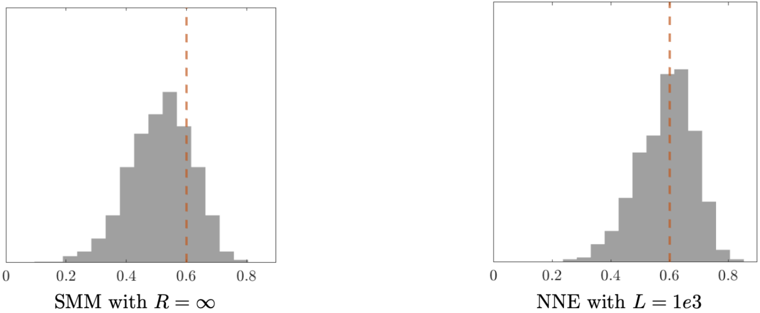
AR(1) Results (cont.)

Table 2: Estimation of AR(1) with Different Moments

Moments	SMM ($R=\infty$)		NNE ($L=1e3$)	
	Bias	RMSE	Bias	RMSE
1) $y_i y_{i-1}$	-0.019 (.003)	0.093 (.002)	-0.017 (.003)	0.091 (.002)
2) $y_i y_{i-1}$, y_i^2	-0.015 (.003)	0.088 (.002)	-0.013 (.003)	0.086 (.002)
3) $y_i y_{i-k}, k=1,2,3$	-0.026 (.003)	0.099 (.003)	-0.015 (.003)	0.091 (.002)
4) $y_i y_{i-k}, k=1,2,...,10$	-0.043 (.003)	0.111 (.003)	-0.014 (.003)	0.095 (.002)
5) $y_i y_{i-1}$, $y_i^2 y_{i-1}$, $y_i y_{i-1}^2$	-0.075 (.003)	0.130 (.003)	-0.014 (.003)	0.095 (.002)
6) $y_i y_{i-k}$, $y_i^2 y_{i-k}$, $y_i y_{i-k}^2$, $k=1,2,3$	-0.086 (.003)	0.135 (.003)	-0.013 (.003)	0.096 (.002)

Notes: SMM ($R=\infty$) is the same as GMM and follows the usual 2-step procedure. The 1st step constructs optimal weighting matrix accounting for serial correlations. NNE ($L=1e3$) uses a shallow neural net of 32 hidden nodes. Reported numbers are based on 1000 Monte Carlo datasets. Numbers in parentheses are standard errors.

AR(1) Results (cont.)



Notes: Histograms of the estimates of β across 1000 Monte Carlo datasets. The vertical dashed red lines display the true value of β. The moment specification follows row 6 in Table 2.

Figure 3: Distributions of Estimates for AR(1) with Redundant Moments

Accuracy and Efficiency

Accuracy:

- NNE recovers (ρ, σ) with low bias across the parameter space
- For $T \geq 100$, performance is close to MLE; gap narrows as $S \to \infty$
- Bhatt, Misra, Qi (2023): NNE RMSE within 5% of MLE benchmark for AR(1)

Efficiency:

- **MLE**: iterative optimization, $\sim$seconds per dataset
- **SMM**: many simulations per objective evaluation, $\sim$minutes
- **NNE**: single forward pass after training, $\sim$milliseconds
- Training cost is *amortized* — paid once, reused for all future estimations
- Speed advantage grows with the number of datasets and model complexity

Roadmap

Choosing Moments: The Key Practical Decision

The NNE is only as good as the moments you feed it.

What makes a good moment?

- **Informative**: variation in the moment maps to variation in θ
- **Low-noise**: estimable reliably from your sample size N
- **Non-redundant**: adds something the other moments don't already capture

Concrete examples from the paper:

Model	Parameter	Identifying moment
AR(1)	ρ	Lag-1 autocorrelation $\hat{\rho}_1$
AR(1)	σ	Variance $\widehat{\mathrm{Var}}(y)$
Consumer search	Search cost c	Mean consideration set size $\mathbb{E}[\lvert\mathcal{C}_i\rvert]$
Consumer search	Quality μ_j	Purchase rate $\Pr(\text{buy } j)$

Practical guidance: more moments help up to a point — adding uninformative ones introduces noise. Use domain knowledge to anchor your choice.

Bhatt, Misra, and Qi (2023)

- Published in *Marketing Science*: "Estimating Parameters of Structural Models Using Neural Networks"
- **Goal**: provide a general-purpose, computationally efficient estimator for structural models with intractable likelihoods
- **Approach**: NNE — simulate (parameter, moment) pairs, train a NN, apply to data
- **Contributions**:
  1. Theoretical properties: consistency and asymptotic normality of NNE as $S \to \infty$ and network width $H \to \infty$
  2. Practical guidelines: architecture choice, training sample size, moment selection
  3. Applications: AR(1) time series, consumer search model, demand estimation
- Paper is available in the `Readings/` folder

Key Definition: Shallow Neural Network

Definition: Shallow Neural Network

A **shallow neural network** with H hidden units is a function $f : \mathbb{R}^K \to \mathbb{R}^P$:

$$f(m) = A^{(2)}\,\sigma\Big(A^{(1)}m + b^{(1)}\Big) + b^{(2)}$$

where $A^{(1)} \in \mathbb{R}^{H\times K}$, $b^{(1)} \in \mathbb{R}^H$, $A^{(2)} \in \mathbb{R}^{P\times H}$, $b^{(2)} \in \mathbb{R}^P$.

- Exactly **one** hidden layer — the simplest NN with universal approximation capacity
- Bhatt et al. establish that the shallow NNE is consistent as $S, H \to \infty$ under mild regularity conditions
- In practice, deeper architectures often perform better; shallow NN serves as a clean theoretical baseline

Consumer Search Model

Setup (sequential search; Weitzman 1979):

- Consumer i considers J products; pays cost c to inspect each one
- Utility from product j: $u_{ij} = \mu_j - p_j + \varepsilon_{ij}$, where ε_{ij} is a match value revealed only upon search
- **Optimal strategy**: inspect products in decreasing order of reservation utility z_j, where z_j solves

$$c = \int_{z_j}^{\infty} (u - z_j)\, dF_\varepsilon(u)$$

- Stop searching when expected gain from the next product falls below c

Parameters: $\theta = (\mu_1, \ldots, \mu_J,\ c,\ \sigma_\varepsilon)$

- Likelihood of observed consideration sets is **intractable** — NNE is especially valuable here

NNE Applied to Consumer Search

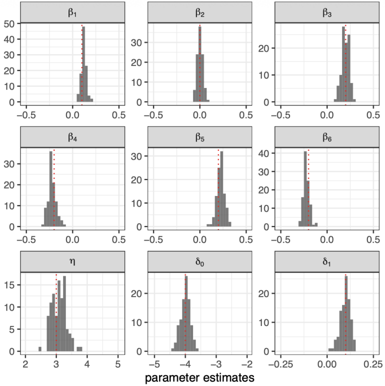
Moments used to train the NNE:

- Product purchase rates: $\Pr(\text{purchase product } j)$
- Mean consideration set size: $\mathbb{E}[|\mathcal{C}_i|]$
- Fraction of consumers who search at least k products, for $k = 1, 2, \ldots$

Results from Bhatt, Misra, Qi (2023):

- NNE recovers search cost c and product qualities $\{\mu_j\}$ with accuracy comparable to full structural estimation
- Speed advantage: $10\times$–$100\times$ faster than simulated MLE
- Performance degrades gracefully as fewer moments are used — robust to moment selection in practice

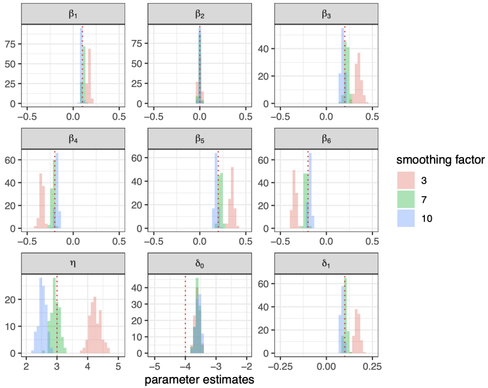
NNE Results for Consumer Search



Notes: Each plot corresponds to one parameter in the search model. The red dotted line shows the true parameter value. The histogram shows the parameter estimates across 100 Monte Carlo datasets. The range of the horizontal axis equals the parameter's range in Θ.

Figure 4: NNE's Estimates for Search Model in Monte Carlo

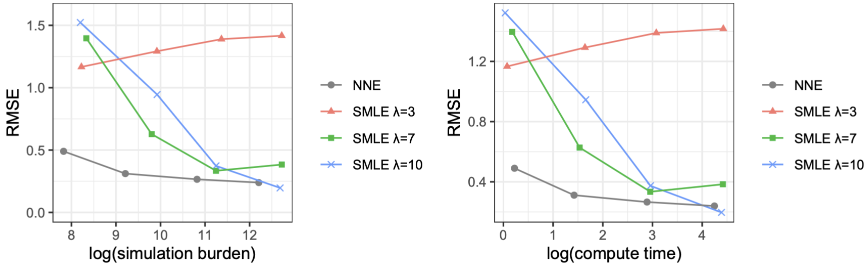
NNE Results for Consumer Search (cont.)



Notes: Same as Figure 4.

Figure 5: SMLE's Estimates for Search Model in Monte Carlo

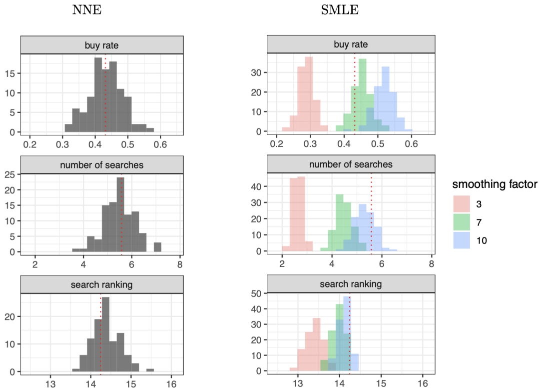
NNE Results for Consumer Search (cont.)



Notes: The "error" in RMSE is defined as the Euclidean distance between the true $\boldsymbol{\theta}$ and its estimate. Simulation burden is the number of search model simulations required to carry out estimation. Simulation burden and compute time are varied by changing L^* and R: $L^* \in \{2500, 1e4, 5e4, 2e5\}$, $R \in \{5, 25, 100, 400\}$. The number of hidden nodes increases with L^* and we set it to be roughly $\propto \sqrt{L}$ (see Section 2.4 and Appendix B).

Figure 6: RMSE vs. Computational Cost for Search Model in Monte Carlo

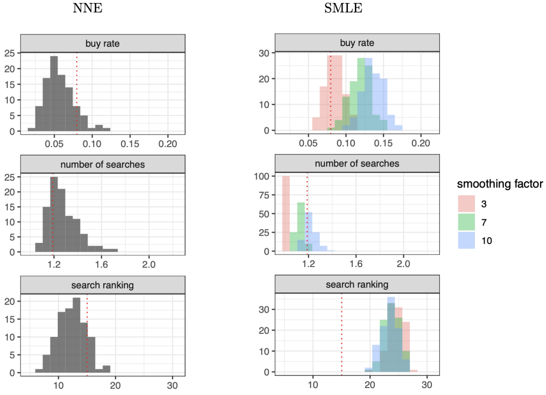
NNE Results for Consumer Search (cont.)



Notes: Plots are based on 100 Monte Carlo datasets (all generated under the true $\boldsymbol{\theta}$). Take the 1st plot as example. On each Monte Carlo dataset, we estimate the search model and calculate the buy rate of a dataset simulated by the estimated search model. The histogram plots the buy rates calculated as such across the 100 Monte Carlo datasets. The red dotted line shows the average of the buy rates directly observed in the Monte Carlo datasets.

Figure 7: Model Fit on Key Statistics in Monte Carlo

NNE Results for Consumer Search (cont.)



Notes: The histograms are based on 100 datasets bootstrapped from the real data. Take the 1st plot as example. On each bootstrapped dataset, we estimate the search model and calculate the buy rate of a dataset simulated by the estimated search model. The histogram plots the buy rates calculated as such across the 100 bootstrapped datasets. The red dotted line shows the buy rate observed in real data.

Figure 8: Model Fit on Key Statistics in Real Data

NNE Results for Consumer Search (cont.)

Table 4: Search Model Estimates on Real Data

	NNE		SMLE ($\lambda = 3$)		SMLE ($\lambda = 7$)		SMLE ($\lambda = 10$)	
Simul. burden ($\times 10^4$)	1.00		9.09		8.28		8.24	
Compute time (mins)	3.83		33.27		30.34		30.25	
Stars, β_1	0.215	(.078)	0.218	(.109)	0.111	(.063)	0.081	(.047)
Review, β_2	-0.089	(.107)	-0.046	(.188)	-0.005	(.106)	0.001	(.076)
Location, β_3	0.111	(.108)	0.178	(.196)	0.080	(.108)	0.056	(.077)
Chain, β_4	0.018	(.093)	-0.012	(.119)	-0.007	(.067)	-0.007	(.049)
Promotion, β_5	0.183	(.091)	0.105	(.108)	0.050	(.062)	0.037	(.045)
Price, β_6	-0.240	(.080)	-0.390	(.117)	-0.208	(.071)	-0.155	(.052)
Outside, η	4.038	(.594)	3.670	(.927)	2.803	(.510)	2.557	(.361)
Search, δ_0	-2.953	(.259)	-0.307	(.260)	-1.290	(.191)	-1.567	(.151)
Search, δ_1	0.049	(.036)	-0.147	(.093)	-0.083	(.067)	-0.061	(.053)

Notes: Numbers in parentheses are asymptotic standard errors (for SMLE) and estimates of statistical accuracy outputted by neural net (for NNE). All reported numbers are averaged across 100 estimation runs on the real data (estimates vary slightly between estimation runs because SMLE and NNE are both simulation-based).

NNE Results for Consumer Search (cont.)

Table 5: NNE in Search Model with Alternative Moment Specifications

Number of moments	Total |bias|	RMSE
16	1.210 (.027)	0.649 (.017)
32	1.201 (.026)	0.630 (.015)
40	0.914 (.029)	0.508 (.015)
46	0.183 (.029)	0.311 (.017)
60	0.186 (.035)	0.291 (.015)
81	0.171 (.033)	0.279 (.015)

Notes: Each row corresponds to one specification of moments. Total |bias| is the sum of absolute biases across the parameters of the search model. Results are based on 100 Monte Carlo datasets. Numbers in parentheses are standard errors.

Roadmap

When Is NNE Useful and Preferable?

NNE is the right tool when:

- The structural model has an **intractable or expensive likelihood**
- Simulation from the model is **fast** relative to inference
- The estimator will be applied to **many datasets** (training cost is amortized across all of them)
- The parameter space is **high-dimensional** (gradient-based optimization of the structural objective is fragile)

Limitations to keep in mind:

- Requires a well-chosen set of **informative moments**
- Training quality depends on **coverage of the prior** $\Pi(\theta)$
- Not ideal for very small samples (moments are too noisy)
- Standard errors require additional work: bootstrap or delta method

Application to My Research

Application to My Research (cont.)

-
-
-